SaveX

Smart kitchen expiry tracker: architecture and solution overview

Hackathon POC | React Native, FastAPI, Whisper, Mistral AI, Supabase, Twilio WhatsApp

1. The idea

Households throw away food mostly because they forget what is in the kitchen and when it expires. SaveX fixes this in three moves.

- **Speak it in.** The user says what they bought, the day, how many days it lasts and where it is kept. Whisper converts speech to text and Mistral turns it into a clean record.
- **Get warned in time.** A daily scheduler tracks every item and alerts the user 7 days (slight), 3 days (moderate) and 1 day (high) before expiry, on WhatsApp and inside the app.
- **Cook what is expiring.** A quick dish planner suggests breakfast, lunch and dinner from items in the kitchen, always using at least one item that is close to expiry.

2. Architecture

The client is now a React Native mobile app. It talks only to FastAPI, which orchestrates Whisper, Mistral, the database and the notification service.

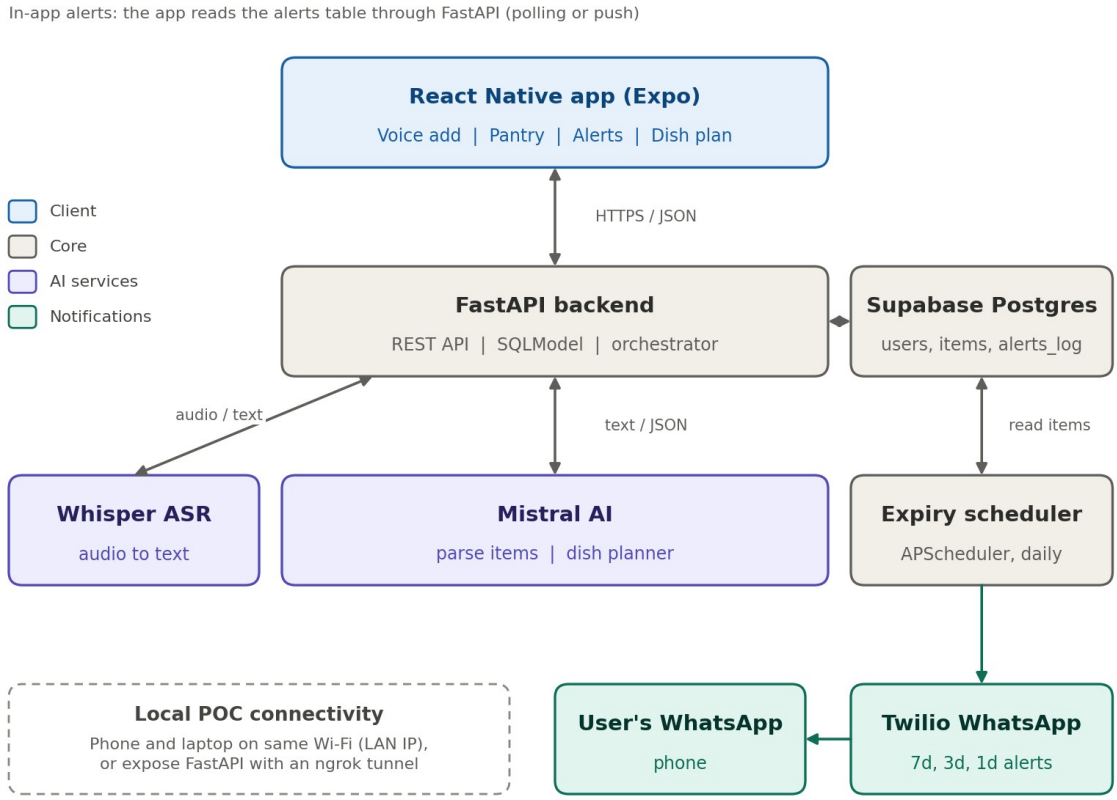


Figure 1. SaveX architecture (mobile-first POC)

What changed from the previous version

- The web app is replaced by a **React Native (Expo)** app. In-app alerts are served by the same FastAPI endpoints.
- A `users` table stores each person's WhatsApp number so the scheduler knows where to send alerts.
- The phone must reach FastAPI running on the laptop, so the POC uses the laptop's LAN address or an ngrok tunnel.

Components

Component	Technology	Responsibility
Mobile app	React Native (Expo)	Records voice, lists pantry items, shows the alerts inbox and dish plan, and asks the user to confirm each parsed item.
Backend	FastAPI + SQLModel	Single orchestrator. Exposes REST endpoints, calls Whisper and Mistral, reads and writes the database.
Speech to text	OpenAI Whisper (API or local faster-whisper)	Converts the recorded audio into a transcript.
LLM	Mistral AI API	Parses the transcript into structured item JSON and generates the dish plan.
Database	Supabase Postgres	Stores users, items and the alerts log.
Scheduler	APScheduler	Runs daily, calculates days left for each item, picks the alert level and triggers notifications.
WhatsApp	Twilio WhatsApp sandbox	Delivers alert messages to the user's phone.

3. Key flows

3.1 Add an item by voice

1. The user taps the mic and says: "Bought one litre milk today, lasts five days, keep in fridge."
2. The app records audio (m4a) and uploads it to `POST /voice`.
3. FastAPI sends the audio to Whisper and receives a transcript.
4. FastAPI sends the transcript and today's date to Mistral, which returns JSON with `name`, `bought_date`, `shelf_days` and `location`. Several items in one sentence return a list.
5. The app shows a confirmation card. The user can fix any field, for example a misheard number.
6. The app calls `POST /items`. FastAPI computes `expiry_date = bought_date + shelf_days` and saves the record.

3.2 Expiry tracking and alerts

The scheduler runs once a day, for example at 8:00 AM. For every active item it calculates days left and applies this ladder.

Days left	Level	In-app display	WhatsApp
7	Slight	Yellow chip and alert entry	Friendly heads-up message
3	Moderate	Orange chip and alert entry	Reminder with suggested action
1 or less	High	Red chip, top banner	Urgent message
Below 0	Expired	Item moved to expired list	None

Before sending, the job checks `alerts_log` so the same item and level never triggers twice. Alerts for one user are grouped into a single WhatsApp message per level to avoid spam.

3.3 Quick dish planner

1. The app calls `POST /plan`.
2. FastAPI loads active items sorted by expiry date and marks those expiring within 3 days as priority items.
3. Mistral receives the list with one rule: each of breakfast, lunch and dinner must use at least one priority item, and may add only common basics such as salt, oil, water and spices.
4. Mistral returns JSON with a dish name, items used, short steps and the priority items covered for each meal.
5. FastAPI validates the rule. If a meal has no priority item, it retries once. The app then shows three meal cards.

4. Data model and API

Tables

Table	Key fields
users	id, name, whatsapp_number, created_at
items	id, user_id, name, bought_date, shelf_days, expiry_date, location, status (active, used, expired), created_at
alerts_log	id, item_id, level (slight, moderate, high), channel (whatsapp, app), sent_at, is_read

Endpoints

Method	Path	Purpose
POST	/voice	Upload audio. Returns the transcript and parsed items for confirmation.
POST	/items	Save confirmed items.
GET	/items	List active items with days left and alert level.
PATCH	/items/{id}	Edit an item or mark it used.
GET	/alerts	Alerts for the in-app inbox and banner.
POST	/plan	Return the breakfast, lunch and dinner plan.

5. React Native app

- **Pantry:** items sorted by expiry, with colour chips for slight, moderate and high.
- **Add by voice:** large mic button, live status (recording, transcribing, parsing) and a confirmation card.
- **Alerts:** inbox with unread badge. For the POC the app polls `GET /alerts` every 30 to 60 seconds while open. Expo push notifications are a later upgrade.
- **Dish plan:** three cards with the dish, items used and the expiring items highlighted.

Audio is recorded with `expo-audio` (or `expo-av` on older SDKs). The m4a format is accepted by the Whisper API.

6. Local POC setup

- Run FastAPI on the laptop with host `0.0.0.0`. The phone, running Expo Go, calls the laptop's LAN IP on the same Wi-Fi. Use ngrok if the networks differ.
- Keep secrets in `.env`: `OPENAI_API_KEY`, `MISTRAL_API_KEY`, `DATABASE_URL` (Supabase), `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_WHATSAPP_FROM`.
- SQLModel creates the tables on startup. Seed one user with a WhatsApp number.

Twilio WhatsApp sandbox rules to plan around

These limits come from Twilio's sandbox documentation and directly affect the demo.

- Only phones that have joined the sandbox can receive messages. Each person sends `join <your sandbox code>` to the sandbox number.
- The sandbox session expires three days after joining, after which the user must join again.
- Free-form messages are allowed only within 24 hours of the user's last message to the sandbox. Outside that window, the sandbox allows only its pre-approved templates, and custom templates are not available.

Demo tip: send any message to the sandbox number shortly before presenting so the alert goes through. If that window lapses, fall back to a pre-approved sandbox template. In-app alerts always work either way.

7. Risks and mitigations

Risk	Mitigation
Whisper mishears numbers or item names	Confirmation card before saving. Add a grocery-word hint to the Whisper prompt.
Mistral returns invalid JSON	Strict JSON instruction, schema validation and one retry.
WhatsApp window or sandbox session expires	Message the sandbox before the demo. Use the pre-approved template as a fallback.
Venue Wi-Fi is unreliable	Use a phone hotspot. Switch to local faster-whisper if the API is unreachable.
Duplicate alerts	Check `alerts_log` for item and level before sending.
Dish plan skips expiring items	Validate the rule on the server and retry once.

8. Build order and demo script

Build order

1. SQLModel tables, Supabase connection, `/items` endpoints.
2. `/voice` with Whisper, then Mistral parsing and the confirmation card.
3. Scheduler, alert ladder and Twilio WhatsApp.
4. Dish planner, then React Native polish.

90-second demo

1. Speak two items into the app and confirm the cards.
2. Use a hidden "simulate date" button to jump the clock forward.
3. Show the WhatsApp alert arriving on a phone, then the in-app banner.
4. Tap Dish plan and show meals built around the expiring items.